

§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解



要求:

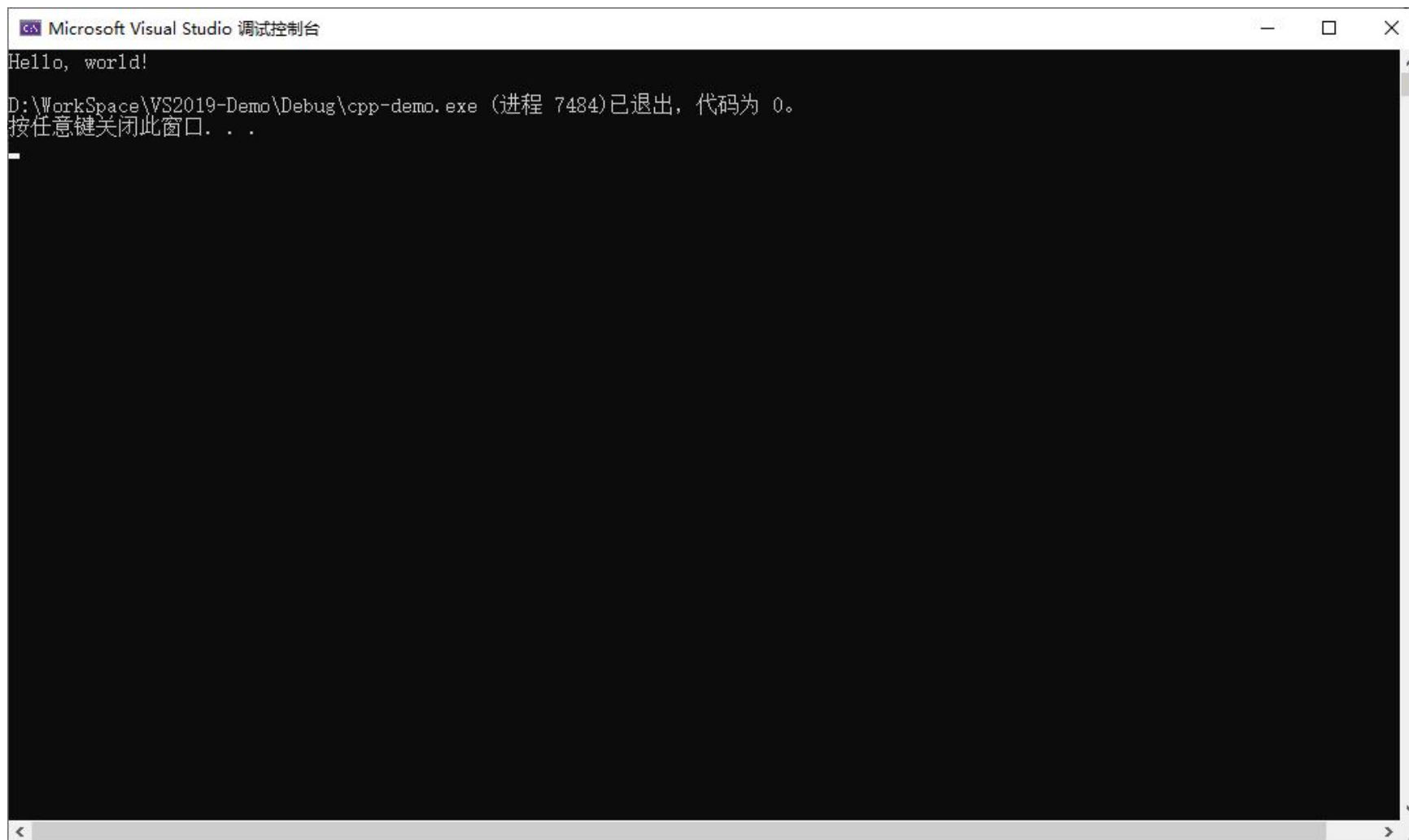
- 1、完成本文档中所有的题目并写出分析、运行结果
- 2、无特殊说明，均使用VS2022编译即可
- 3、直接在本文件上作答，**写出答案/截图（不允许手写、手写拍照截图）**即可；填写答案时，为适应所填内容或贴图，**允许调整**页面的字体大小、颜色、文本框的位置等
 - ★ 贴图要有效部分即可，不需要全部内容
 - ★ 在保证一页一题的前提下，具体页面布局可以自行发挥，简单易读即可
 - ★ **不允许**手写在纸上，再拍照贴图
 - ★ **允许**在各种软件工具上完成（不含手写），再截图贴图
- 4、转换为pdf后提交
- 5、**3月14日前**网上提交本次作业（在“文档作业”中提交）

§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解

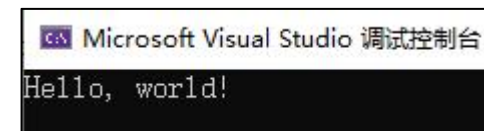


贴图要求：只需要截取输出窗口中的有效部分即可，如果全部截取/截取过大，则视为无效贴图

例：无效贴图

A screenshot of the Microsoft Visual Studio debug console window. The window is titled "Microsoft Visual Studio 调试控制台". The output text is: "Hello, world!", "D:\Workspace\VS2019-Demo\Debug\cpp-demo.exe (进程 7484)已退出, 代码为 0.", and "按任意键关闭此窗口. . .". The window is large, showing the full output area.

例：有效贴图

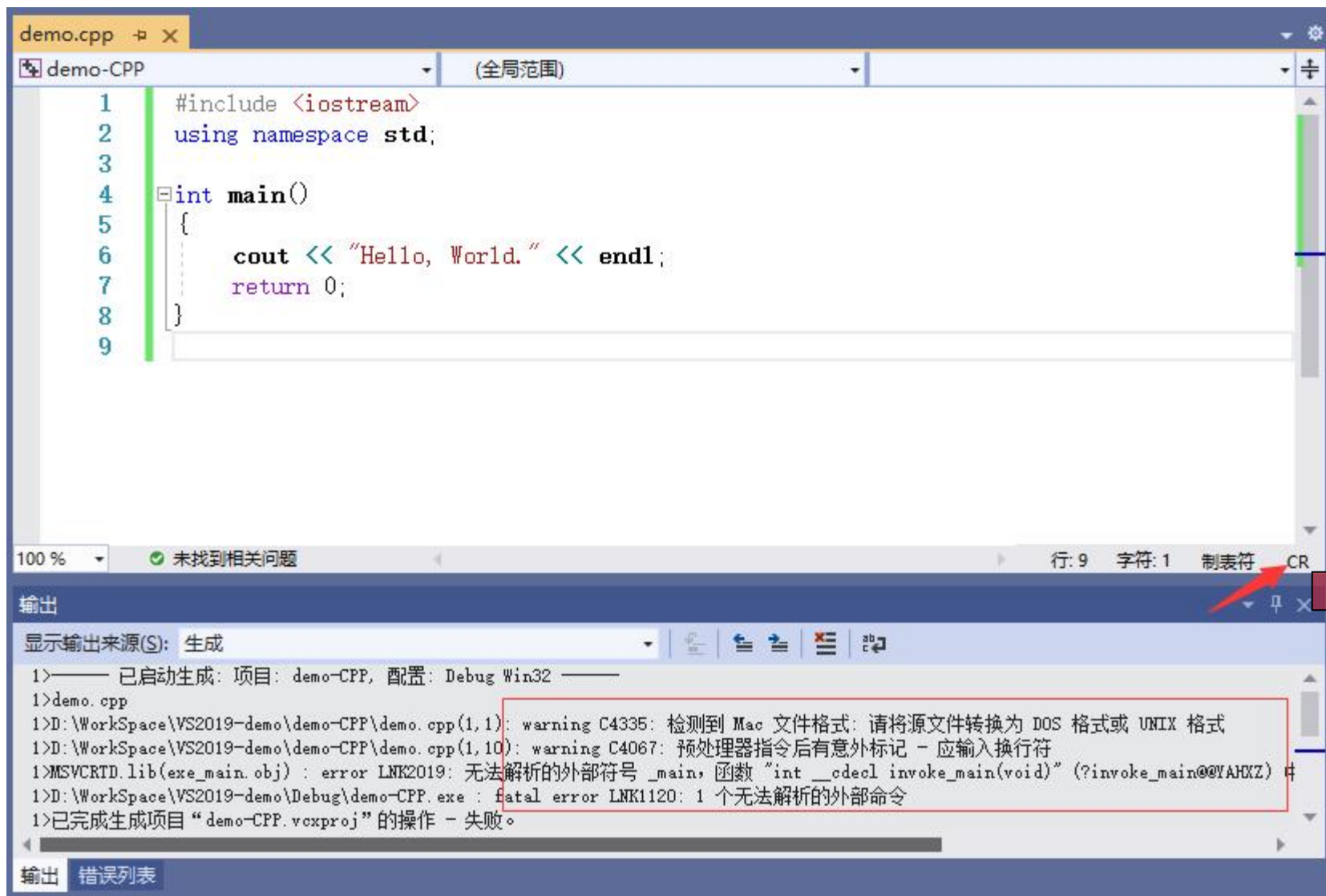
A screenshot of the Microsoft Visual Studio debug console window, showing only the first line of output: "Hello, world!". The window is titled "Microsoft Visual Studio 调试控制台". This is an example of a valid screenshot.

§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解



附：用WPS等其他第三方软件打开PPT，将代码复制到VS2022中后，如果出现类似下面的**编译报错**，则观察源程序编辑窗

的右下角是否为CR，如果是，单击CR，在弹出中选择CRLF，再次CTRL+F5运行即可



§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解



基础知识：用于看懂float型数据的内部存储格式的程序如下：

注意：除了对黄底红字的具体值进行改动外，其余部分不要做改动，也暂时不需要弄懂为什么（需要第6章的知识才能弄懂）

```
#include <iostream>
using namespace std;
int main()
{
    float f = 123.456;
    unsigned char* p = (unsigned char*)&f;
    cout << hex << (int)(*p) << endl;
    cout << hex << (int)*(p+1) << endl;
    cout << hex << (int)*(p+2) << endl;
    cout << hex << (int)*(p+3) << endl;
    return 0;
}
```

//注：忽略本题出现的warning

Microsoft
79
e9
f6
42

上例解读：单精度浮点数123.456，在内存中占四个字节，四个字节的值依次为0x42 0xf6 0xe9 0x79（按打印顺序逆向取）

转换为32bit则为：0100 0010 1111 0110 1110 1001 0111 1001

符号位

8位指数

23位尾数

§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解



基础知识：用于看懂double型数据的内部存储格式的程序如下：

注意：除了对黄底红字的具体值进行改动外，其余部分不要做改动，也暂时不需要弄懂为什么（需要第6章的知识才能弄懂）

```
#include <iostream>
using namespace std;
int main()
{
    double d = 1.23e4;
    unsigned char* p = (unsigned char*)&d;
    cout << hex << (int)(*p) << endl;
    cout << hex << (int)*(p+1) << endl;
    cout << hex << (int)*(p+2) << endl;
    cout << hex << (int)*(p+3) << endl;
    cout << hex << (int)*(p+4) << endl;
    cout << hex << (int)*(p+5) << endl;
    cout << hex << (int)*(p+6) << endl;
    cout << hex << (int)*(p+7) << endl;
    return 0;
}
```

Microsoft
0
0
0
0
0
6
c8
40

上例解读：双精度浮点数1.23e4，在内存中占八个字节，八个字节的值依次为0x40 0xc8 0x06 0x00 0x00 0x00 0x00 0x00(逆向)

转换为64bit则为：0100 0000 1100 1000 0000 0100 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

符号位

11位指数

52位尾数

§ . 基础知识题 – 浮点数机内存储格式(IEEE 754)理解



自学内容：自行以“IEEE754” / “浮点数存储格式” / “浮点数存储原理” / “浮点数存储方式”等关键字，在网上搜索相关文档，读懂并了解浮点数的内部存储机制

学长们推荐的网址：

<https://baike.baidu.com/item/IEEE%20754/3869922?fr=aladdin>

<https://zhuanlan.zhihu.com/p/343033661>

https://www.bilibili.com/video/BV1iW41ld7hd?is_story_h5=false&p=4&share_from=ugc&share_medium=android&share_plat=android&share_session_id=e12b54be-6ffa-4381-9582-9d5b53c50fb3&share_source=QQ&share_tag=s_i×tamp=1662273598&unique_k=AuouME0

https://blog.csdn.net/gao_zhennan/article/details/120717424

<https://www.h-schmidt.net/FloatConverter/IEEE754.html>



§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解

例: float型数的机内表示

格式要求: 多字节时, 每8bit中间加一个空格或- (例: "11010100 00110001" 或 "11010100-00110001")

例1: 100.25

下面是float机内存储手工转十进制的方法:

(1) 得到的32bit的机内表示是: 0100 0010 1100 1000 1000 0000 0000 0000 (42 c8 80 00)

(2) 其中: 符号位是 0

指数是 1000 0101 (填32bit中的原始形式)

指数转换为十进制形式是 133 (32bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是 6 (32bit中的原始形式按IEEE754的规则转换)

1000 0101

- 0111 1111

= 0000 0110 (0x06 = 6)

尾数是 100 1000 1000 0000 0000 0000 (填32bit中的原始形式)

尾数转换为十进制小数形式是 0.56640625 (32bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是 1.56640625 (加整数部分的1后)

100 1000 1000 0000 0000 0000 = $2^0 + 2^{-1} + 2^{-4} + 2^{-8}$

= 0.5 + 0.0625 + 0.00390625 = 0.56640625 => 加1 => 1.56640625

1.56640625 x 2^6 = 100.25 (此处未体现出误差)

下面是十进制手工转float机内存储的方法:

100 = 0110 0100 (整数部分转二进制为7位, 最前面的0只是为了8位对齐, 可不要)

0.25 = 01 (小数部分转二进制为2位)

100.25 = 0110 0100.01 = 1.1001 0001 x 2^6 (确保整数部分为1, 移6位)

符号位: 0

阶码: 6 + 127 = 133 = 1000 0101

尾数(舍1): 1001 0001 => 1001 0001 0000 0000 0000 000 (补齐23位, 后面补14个蓝色的0)

100 1000 1000 0000 0000 0000 (从低位开始四位一组, 共23位)

注意:

- 1、作业中绿底/黄底文字/截图可不填
- 2、计算结果可借助第三方工具完成, 没必要完全手算

本页不用作答



§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解

例: float型数的机内表示

格式要求: 多字节时, 每8bit中间加一个空格或- (例: "11010100 00110001" 或 "11010100-00110001")

例2: 1.2

下面是float机内存储手工转十进制的方法:

(1) 得到的32bit的机内表示是: 0011 1111 1001 1001 1001 1001 1001 1010 (3f 99 99 9a)

(2) 其中: 符号位是 0

指数是 0111 1111 (填32bit中的原始形式)

指数转换为十进制形式是 127 (32bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是 0 (32bit中的原始形式按IEEE754的规则转换)

0111 1111

- 0111 1111

= 0000 0000 (0x0 = 0)

尾数是 001 1001 1001 1001 1001 1010 (填32bit中的原始形式)

尾数转换为十进制小数形式是 0.2000000476837158203125 (32bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是 1.2000000476837158203125 (加整数部分的1后)

001 1001 1001 1001 1001 1010 = $2^{-3} + 2^{-4} + 2^{-7} + 2^{-8} + 2^{-11} + 2^{-12} + 2^{-15} + 2^{-16} + 2^{-19} + 2^{-20} + 2^{-22}$

= 0.125 + ... + 0.0000002384185791015625 (详见右侧蓝色) = 0.2000000476837158203125

=> 加1 = 1.2000000476837158203125 (此处已体现出误差)

下面是十进制手工转float机内存储的方法:

1 = 1 (整数部分转二进制为1位)

0.2 = 0011 0011 0011 0011 0011 0011 (小数部分无限循环, 转为二进制的24位)

=> 0011 0011 0011 0011 0011 010 (四舍五入为23位, 此处体现出误差)

1.2 = 1.0011 0011 0011 0011 0011 010 = 1.0011 0011 0011 0011 0011 010 x 2^0 (确保整数部分为1, 移0位)

符号位: 0

阶码: 0 + 127 = 127 = 0111 1111

尾数(舍1): 0011 0011 0011 0011 0011 010 (共23位)

001 1001 1001 1001 1001 1010 (从低位开始四位一组, 共23位)

注意:

- 1、作业中绿底/黄底文字/截图可不填
- 2、计算结果可借助第三方工具完成, 没必要完全手算

0.125 +
0.0625 +
0.0078125 +
0.00390625 +
0.00048828125 +
0.000244140625 +
0.000030517578125 +
0.0000152587890625 +
0.0000019073486328125 +
0.00000095367431640625 +
0.0000002384185791015625

0.2000000476837158203125

本页不用作答



§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解

1、float型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

A. 1234567.7654321 (此处设学号是1234567，需换成本人学号，小数为学号逆序，非本人学号0分，下同!!!)

注：尾数为正、指数为正

(1) 得到的32bit的机内表示是：__0100 1010 0000 1111 0100 0000 1000 0011__ (不是手算，用P.4方式打印)

(2) 其中：符号位是____0____

指数是__1001 0100_ (填32bit中的原始形式)

指数转换为十进制形式是__148__ (32bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是__21__ (32bit中的原始形式按IEEE754的规则转换)

尾数是____000 1111 0100 0000 1000 0011____ (填32bit中的原始形式)

尾数转换为十进制小数形式是__0.11915624141693115__ (32bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是__1.11915624141693115__ (加整数部分的1)

注：转换为十进制小数用附加的工具去做，自己去网上找工具也行，但要满足精度要求 (下同!!!)



§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解

1、float型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

B. -7654321.1234567 (设学号为1234567，按规则更换为学号和学号逆序)

注：尾数为负、指数为正

(1) 得到的32bit的机内表示是：___1100 1010 0000 0111 1100 0000 1000 0011___ (不是手算，用P.4方式打印)

(2) 其中：符号位是___1___

指数是___1001 0100___ (填32bit中的原始形式)

指数转换为十进制形式是___148___ (32bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是___21___ (32bit中的原始形式按IEEE754的规则转换)

尾数是___110 1001 1001 0111 0110 0010___ (填32bit中的原始形式)

尾数转换为十进制小数形式是___-0.06056249141693115___ (32bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是___-1.06056249141693115___ (加整数部分的1)



§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解

1、float型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

C. 0.001234567 (设学号为1234567，按规则更换为学号和学号逆序)

注：尾数为正、指数为负

(1) 得到的32bit的机内表示是：____0011 1011 0001 1010 0001 0101 1000 0100____ (不是手算，用P.4方式打印)

(2) 其中：符号位是____0____

指数是____0111 0110____ (填32bit中的原始形式)

指数转换为十进制形式是____118____ (32bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是____-9____ (32bit中的原始形式按IEEE754的规则转换)

尾数是____001 1010 0001 0101 1000 0100____ (填32bit中的原始形式)

尾数转换为十进制小数形式是____0.2037816047668457____ (32bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是____1.2037816047668457____ (加整数部分的1)



§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解

1、float型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

D. -0.007654321 (设学号为1234567，按规则更换为学号和学号逆序)

注：尾数为负、指数为负

(1) 得到的32bit的机内表示是：___1011 1011 0001 1010 0001 0101 1000 0100___ (不是手算，用P.4方式打印)

(2) 其中：符号位是___1___

指数是___0111 0110___ (填32bit中的原始形式)

指数转换为十进制形式是___118___ (32bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是___-9___ (32bit中的原始形式按IEEE754的规则转换)

尾数是___00110100001010110000100___ (填32bit中的原始形式)

尾数转换为十进制小数形式是___-0.2037816047668457___ (32bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是___-1.2037816047668457___ (加整数部分的1)



§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解

2、double型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

A. 1234567.7654321 (设学号为1234567，按规则更换为学号和学号逆序)

注：尾数为正、指数为正

(1) 得到的64bit的机内表示是：____0100 0001 0100 0001 1111 0000 0001 0000 0101 0000 1100 1001 1010 0000 1100 1000____ (不是手算，用P.5方式打印)

(2) 其中：符号位是____0____

指数是__10000010100__ (填64bit中的原始形式)

指数转换为十进制形式是__1044__ (64bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是__17__ (64bit中的原始形式按IEEE754的规则转换)

尾数是____0001 1111 0000 0001 0000 0101 0000 1100 1001 1010 0000 1100 1000____ (填64bit中的原始形式)

尾数转换为十进制小数形式是__0.1211093097463607__ (64bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是__1.1211093097463607__ (加整数部分的1)



§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解

2、double型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

B. -7654321.1234567 (设学号为1234567，按规则更换为学号和学号逆序)

注：尾数为负、指数为正

(1) 得到的64bit的机内表示是：____1100 0001 0100 0001 1111 0000 0001 0000 0101 0000 1100 1001 1010 0000 1100 1000____ (不是手算，用P.5方式打印)

(2) 其中：符号位是____1____

指数是____100 0001 0100____ (填64bit中的原始形式)

指数转换为十进制形式是____1044____ (64bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是____17____ (64bit中的原始形式按IEEE754的规则转换)

尾数是____0001111100000001000001010000110010011010000011001000____ (填64bit中的原始形式)

尾数转换为十进制小数形式是____-0.1211093097463607____ (64bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是____-0.1211093097463607____ (加整数部分的1)



§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解

2、double型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

C. 0.001234567 (设学号为1234567，按规则更换为学号和学号逆序)

注：尾数为正、指数为负

(1) 得到的64bit的机内表示是：_____0011 1111 0110 0011 0100 0010 1011 0000 1000 0111 0100 1111
0111 0010 1011 1000_____ (不是手算，用P.5方式打印)

(2) 其中：符号位是_____0_____

指数是_____011 1111 0110_____ (填64bit中的原始形式)

指数转换为十进制形式是_____1014_____ (64bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是_____ -13 _____ (64bit中的原始形式按IEEE754的规则转换)

尾数是_____0011 0100 0010 1011 0000 1000 0111 0100 1111 0111 0010 1011 1000_____ (填
64bit中的原始形式)

尾数转换为十进制小数形式是_____0.2037816320000001_____ (64bit中的原始形式按二进制原码
形式转换)

尾数表示的十进制小数形式是_____1.2037816320000001_____ (加整数部分的1)



§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解

2、double型数的机内表示

格式要求：多字节时，每4bit中间加一个空格或- (例：“1101 0100 0011 0001” 或 “1101-0100-0011-0001”)

D. -0.007654321 (设学号为1234567，按规则更换为学号和学号逆序)

注：尾数为负、指数为负

(1) 得到的64bit的机内表示是：_____1011 1111 0110 0011 0100 0010 1011 0000 1000 0111 0100 1111
0111 0010 1011 1000_____ (不是手算，用P.5方式打印)

(2) 其中：符号位是___1___

指数是___011 1111 0110_____ (填64bit中的原始形式)

指数转换为十进制形式是___1014_____ (64bit中的原始形式按二进制原码形式转换)

指数表示的十进制形式是___-13_____ (64bit中的原始形式按IEEE754的规则转换)

尾数是__0011 0100 0010 1011 0000 1000 0111 0100 1111 0111 0010 1011 1000_____ (填64bit中的原始形式)

尾数转换为十进制小数形式是___-0.2037816320000001_____ (64bit中的原始形式按二进制原码形式转换)

尾数表示的十进制小数形式是___-1.2037816320000001_____ (加整数部分的1)

§. 基础知识题 – 浮点数机内存储格式(IEEE 754)理解



3、总结

(1) float型数据的32bit是如何分段来表示一个单精度的浮点数的？给出bit位的分段解释

尾数的正负如何表示？尾数如何表示？指数的正负如何表示？指数如何表示？

32bit分为三部分，第一个bit作为符号位，紧跟的8bit代表指数，最后23bit表示尾数

正负看第一位，0正1负。将23bit作为小数部分可算出尾数小数部分。指数通过转为ieee754可知正负。指数依靠8bit转为十进制。

(2) 为什么float型数据只有7位十进制有效数字？为什么最大只能是 3.4×10^{38} ？

有些资料上说有效位数是6~7位，能找出6位/7位不同的例子吗？

Float型数据只有7位十进制有效数字是由于IEEE 754标准中规定了单精度浮点数（32位）的存储结构，其中一部分用于表示指数，另一部分用于表示尾数。对于单精度浮点数，最大的指数为 $2^8 - 1 = 255$ （指数位全为1），这意味着最大的指数为 2^{127} 。由于指数部分是带符号的，所以最大值为 3.4×10^{38} 。

例子，六位：0.123456 七位：12.34567

(3) double型数据的64bit是如何分段来表示一个双精度的浮点数的？给出bit位的分段解释

尾数的正负如何表示？尾数如何表示？指数的正负如何表示？指数如何表示？

见最后一页

(4) 为什么double型数据只有15位十进制有效数字？为什么最大只能是 1.7×10^{308} ？

有些资料上说有效位数是15~16位，能找出15位/16位不同的例子吗？

双精度浮点数最大精度为15-16位，这是因为浮点数的表示方式决定了其精度。在IEEE 754标准中，双精度浮点数的尾数部分有52位，这意味着它最多可以表示52位的二进制数。当将这些二进制数转换为十进制时，大约只能保留15-16位有效数字。

双精度浮点数的指数部分有11位，因此可以表示的指数范围为 $[-1022, 1023]$ 。这意味着浮点数可以表示的数量级大约是10的 ± 308 次方。

15位有效数字的例子：123456789012345

16位有效数字的例子：1234567890123456

注：



§. 基础知识题 - 浮点数机内存储格式(IEEE 754)理解

4、思考

(1) 8/11bit的指数的表示形式是2进制补码吗？如果不是，一般称为什么方式表示？

不是。在浮点数的表示中，通常采用的是偏移二进制补码表示法。

(2) double赋值给float时，下面两个程序，double型常量不加F的情况下，左侧有warning，右侧无warning，为什么？

总结一下规律

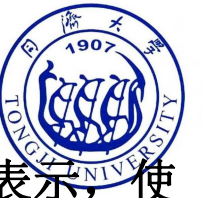
```
#include <iostream>
using namespace std;
int main()
{
    float f = 1.2;
    unsigned char* p = (unsigned char*)&f;
    cout << hex << (int)(*p) << endl;
    cout << hex << (int)*(p+1) << endl;
    cout << hex << (int)*(p+2) << endl;
    cout << hex << (int)*(p+3) << endl;
    return 0;
}
```

warning C4305: “初始化”: 从“double”到“float”截断

```
#include <iostream>
using namespace std;
int main()
{
    float f = 100.25;
    unsigned char* p = (unsigned char*)&f;
    cout << hex << (int)(*p) << endl;
    cout << hex << (int)*(p+1) << endl;
    cout << hex << (int)*(p+2) << endl;
    cout << hex << (int)*(p+3) << endl;
    return 0;
}
```

因为左边是高精度向低精度转化，由于字节长度不同，会导致字节截断而精度降低产生warning。而右边则是通过补充字节的方式变成double，因而不会有影响。

总结：高精度变低精度，会截断warning。低精度变高精度，不会截断warning。



3. (3) 双精度的浮点数使用64位来表示。这64位可以分为三个部分：符号位、指数部分和尾数部分
符号位（1位）：决定浮点数的正负。0表示正数，1表示负数。

指数部分（11位）：指数部分用来表示浮点数的数量级，使用偏移二进制补码表示。通过偏移二进制补码表示，使得指数可以表示负数。指数部分的范围为 $[-1022, 1023]$ ，但是使用偏移二进制补码表示，实际的指数范围是 $[-1022, 1023]$ 。

尾数部分（52位）：尾数部分用来表示浮点数的精度。尾数部分是一个小数，可以是正数也可以是负数。它以二进制小数的形式表示，小数点左边的部分默认为1（因为规范化），但不存储，只存储小数点后的位数。因此，尾数部分的范围大致是 $[1.0, 2.0)$ 。

尾数的正负由符号位表示，指数的正负由偏移二进制补码表示。如果指数部分为全0，则表示浮点数是非规范化数或者零。如果指数部分全为1，则表示浮点数是特殊值（如无穷大或NaN）。